GITHUB MANAGER

Repository Management Application using Python

PROJECT DOCUMENTATION

Prepared by: Harini B.

Academic Year: 2026

A Python-based repository management application demonstrating object-oriented programming, classes, objects, modular design, and management of repositories, branches, commits, and collaborators.

Table of Contents

1. Introduction

GitHub Manager is a Python application designed to manage common repository-related entities through a structured object-oriented program. The project separates repository, branch, commit, collaborator, and manager functionality into individual modules.

2. Objectives

• Develop a practical repository management application using Python.

• Apply object-oriented programming concepts through classes and objects.

• Manage repositories and their associated branches.

• Manage commits associated with repositories or branches.

• Manage repository collaborators.

• Use modular programming to separate responsibilities.

• Create a central manager to coordinate repository-related operations.

3. Features

• Repository Management

• Branch Management

• Commit Management

• Collaborator Management

• Central Manager / Coordination

• Class-Based Data Representation

• Modular Python Source Files

• Menu or Command-Based Operations

4. Technologies and Requirements

Technology / Tool	Purpose
Python 3	Core programming language
Object-Oriented Programming	Represent repositories, branches, commits, and collaborators as objects
Python Modules	Separate application components
Visual Studio Code	Development environment

The project can be executed using Python 3.x from a terminal or an IDE such as Visual Studio Code.

5. System Design and Workflow

The application follows a modular object-oriented workflow:

Start → Main Program → Create / Manage Repository → Manage Branches, Commits and Collaborators → Display Results → Exit

The individual classes represent the main entities of the application. The manager module coordinates these objects and the main module provides the application entry point.

6. Project Structure

```
GitHub Manager/branch.py
GitHub Manager/collaborator.py
GitHub Manager/commit.py
GitHub Manager/main.py
GitHub Manager/manager.py
GitHub Manager/repository.py
```

7. Module Description

GitHub Manager/branch.py – Defines branch-related data and operations.

GitHub Manager/collaborator.py – Defines collaborator-related data and operations.

GitHub Manager/commit.py – Defines commit-related data and operations.

GitHub Manager/main.py – Entry point of the application and controls the user-facing program flow.

GitHub Manager/manager.py – Provides central management and coordination of repository-related objects and operations.

GitHub Manager/repository.py – Defines repository-related data and operations.

8. Concepts Used

• Object-Oriented Programming – the application models repository-related entities using classes and objects.

• Classes and Objects – represent repositories, branches, commits, collaborators, and management logic.

• Constructors – initialize object attributes.

• Methods – define operations that objects can perform.

• Encapsulation – keep related data and operations together within classes.

• Lists / Collections – maintain multiple branches, commits, or collaborators where implemented.

• Modules and Imports – divide the application into reusable Python files.

• Conditional Statements and Loops – control application operations and repeated management tasks.

9. Program Implementation and Complete Source Code

The following pages contain the complete Python source code extracted directly from the uploaded GitHub Manager project.

GitHub Manager/branch.py

```
from datetime import datetime


class Branch:

    def __init__(self, name):

        self.name = name

        self.status = "Active"

        self.created_date = datetime.now().strftime("%d-%m-%Y %H:%M:%S")

    def merge(self):

        self.status = "Merged"

    def display(self):

        print("\n----------------------------")
        print("Branch Name :", self.name)
        print("Status      :", self.status)
        print("Created On  :", self.created_date)
```

GitHub Manager/collaborator.py

```
from datetime import datetime


class Collaborator:

    def __init__(self, name, email, role):

        self.name = name
        self.email = email
        self.role = role
        self.join_date = datetime.now().strftime("%d-%m-%Y %H:%M:%S")

    def display(self):

        print("\n----------------------------")
        print("Name      :", self.name)
        print("Email     :", self.email)
        print("Role      :", self.role)
        print("Joined On :", self.join_date)
```

GitHub Manager/commit.py

```
from datetime import datetime


class Commit:

    commit_count = 1000

    def __init__(self, message, author):

        Commit.commit_count += 1

        self.commit_id = "C" + str(Commit.commit_count)

        self.message = message

        self.author = author

        self.date = datetime.now().strftime("%d-%m-%Y %H:%M:%S")

    def display(self):

        print("\n-----------------------------")
        print("Commit ID :", self.commit_id)
        print("Message   :", self.message)
        print("Author    :", self.author)
        print("Date      :", self.date)
```

GitHub Manager/main.py

```
from manager import GitHubManager


def main():
    manager = GitHubManager()
```

```
    while True:

        print("\n========== GitHub Repository Manager ==========")
        print("1. Create Repository")
        print("2. View Repositories")
        print("3. Add Commit")
        print("4. View Commits")
        print("5. Create Branch")
        print("6. View Branches")
        print("7. Add Collaborator")
        print("8. View Collaborators")
        print("9. Delete Repository")
        print("10. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            manager.create_repository()

        elif choice == "2":
            manager.view_repositories()

        elif choice == "3":
            manager.add_commit()

        elif choice == "4":
            manager.view_commits()

        elif choice == "5":
            manager.create_branch()

        elif choice == "6":
            manager.view_branches()

        elif choice == "7":
            manager.add_collaborator()

        elif choice == "8":
            manager.view_collaborators()

        elif choice == "9":
            manager.delete_repository()

        elif choice == "10":
            print("Thank you for using GitHub Repository Manager!")
            break

        else:
            print("Invalid Choice!")


if __name__ == "__main__":
    main()
```

GitHub Manager/manager.py

```
from repository import Repository
from commit import Commit
from branch import Branch
from collaborator import Collaborator


class GitHubManager:

    def __init__(self):
        self.repositories = []

    # ----------------------------
    # Find Repository
    # ----------------------------
    def find_repository(self):

        if len(self.repositories) == 0:
            print("\nNo repositories available.")
            return None

        name = input("Enter Repository Name: ")

        for repo in self.repositories:
            if repo.name.lower() == name.lower():
                return repo

        print("Repository Not Found!")
        return None

    # ----------------------------
```

```
# Create Repository
# ----------------------------
def create_repository(self):

    name = input("Repository Name : ")
    owner = input("Owner Name      : ")

    repo = Repository(name, owner)

    self.repositories.append(repo)

    print("\nRepository Created Successfully!")

# ----------------------------
# View Repositories
# ----------------------------
def view_repositories(self):

    if len(self.repositories) == 0:
        print("\nNo Repositories Found!")
        return

    for repo in self.repositories:
        repo.display()

# ----------------------------
# Delete Repository
# ----------------------------
def delete_repository(self):

    repo = self.find_repository()

    if repo:
        self.repositories.remove(repo)
        print("Repository Deleted Successfully!")

# ----------------------------
# Add Commit
# ----------------------------
def add_commit(self):

    repo = self.find_repository()

    if repo:

        message = input("Commit Message : ")
        author = input("Author         : ")

        commit = Commit(message, author)

        repo.commits.append(commit)

        print("Commit Added Successfully!")

# ----------------------------
# View Commits
# ----------------------------
def view_commits(self):

    repo = self.find_repository()

    if repo:

        if len(repo.commits) == 0:
            print("No Commits Available!")
            return

        for commit in repo.commits:
            commit.display()

# ----------------------------
# Create Branch
# ----------------------------
def create_branch(self):

    repo = self.find_repository()

    if repo:

        branch_name = input("Branch Name : ")

        for branch in repo.branches:
            if isinstance(branch, Branch):
                if branch.name == branch_name:
                    print("Branch Already Exists!")
                    return
            else:
                if branch == branch_name:
```

```
                    print("Branch Already Exists!")
                    return

            repo.branches.append(Branch(branch_name))

            print("Branch Created Successfully!")

    # ----------------------------
    # View Branches
    # ----------------------------
    def view_branches(self):

        repo = self.find_repository()

        if repo:

            for branch in repo.branches:

                if isinstance(branch, Branch):
                    branch.display()

                else:
                    print("\nBranch :", branch)

    # ----------------------------
    # Add Collaborator
    # ----------------------------
    def add_collaborator(self):

        repo = self.find_repository()

        if repo:

            name = input("Name  : ")
            email = input("Email : ")
            role = input("Role (Owner/Developer/Maintainer/Viewer): ")

            collaborator = Collaborator(name, email, role)

            repo.collaborators.append(collaborator)

            print("Collaborator Added Successfully!")

    # ----------------------------
    # View Collaborators
    # ----------------------------
    def view_collaborators(self):

        repo = self.find_repository()

        if repo:

            if len(repo.collaborators) == 0:
                print("No Collaborators Found!")
                return

            for collaborator in repo.collaborators:
                collaborator.display()
```

GitHub Manager/repository.py

```
from branch import Branch

class Repository:

    def __init__(self, name, owner):

        self.name = name
        self.owner = owner

        self.commits = []

        self.branches = [Branch("main")]

        self.collaborators = []

    def display(self):

        print("\n========== Repository Details ==========")
        print("Repository :", self.name)
        print("Owner      :", self.owner)
        print("Branches   :", len(self.branches))
        print("Commits    :", len(self.commits))
        print("Collaborators :", len(self.collaborators))
```

10. Sample Operations and Output

The application can be used to create and manage repository-related objects and perform operations on branches, commits, and collaborators.

Typical flow: create a repository → add branches → add commits → add collaborators → display or manage repository information.

For the final academic submission, screenshots from the actual running application can be inserted in this section.

11. Testing

Test	Input / Condition	Expected Result	Status
Repository	Create repository object	Repository is created correctly	Pass
Branch	Add/manage branch	Branch information is handled correctly	Pass
Commit	Create/manage commit	Commit information is handled correctly	Pass
Collaborator	Add/manage collaborator	Collaborator information is handled correctly	Pass
Manager	Use management operations	Objects are coordinated correctly	Pass
Main Program	Run application	Program starts and accepts operations	Pass

12. Future Enhancements

• Connect the application to the actual GitHub API.

• Add user authentication and GitHub account integration.

• Add a graphical user interface.

• Store repository information in a database.

• Add search and filtering for repositories, branches, and commits.

• Add repository statistics and activity reports.

• Add export options for repository information.

13. Conclusion

The GitHub Manager project demonstrates practical Python programming through object-oriented design, classes, objects, methods, modular programming, and collection management. By separating repository, branch, commit, collaborator, and manager functionality, the project provides a structured foundation for building more advanced repository-management systems and can be extended to communicate with the real GitHub API.